

最適化と勾配降下法

fit() の中身を開ける

rkskmt

1

fit() の中では何が起こっているのか

- 分類の回からずっと、この1行に頼ってきた

```
1 model.fit(X, y) # ← この中で何が起こっている？
```

Python

- **fit()** の仕事：パラメータを動かして、**損失（ダメさの点数）が最小になる場所**を探ること
- つまり中身の正体は**最適化**

① 今回の問い

「関数の最小値を探す」を、機械はどうやってやるのか？

2

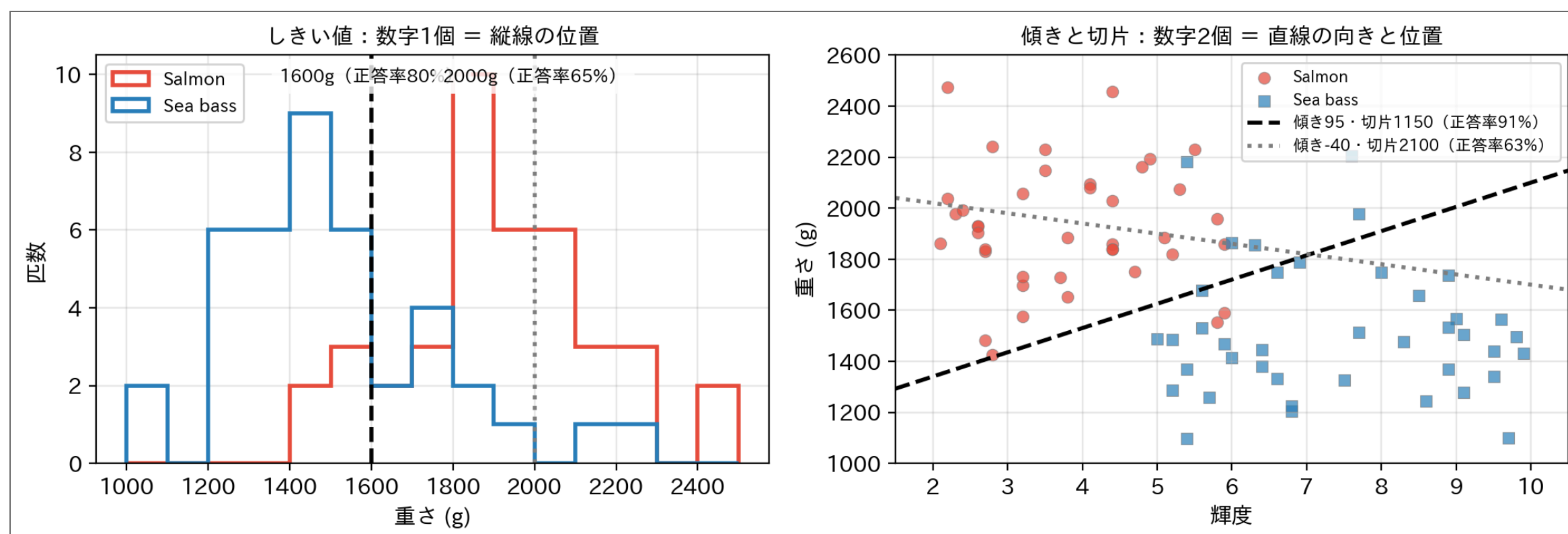
パラメータとは？

- モデルの中にある、**自由に動かせる数字**のこと
- 実は、これまでの講義はずっと「パラメータを選ぶ作業」だった

回	モデル	パラメータ
1次元分類	しきい値で判定	しきい値（数字1個）
2次元の決定境界	直線で判定	傾きと切片（数字2個）
クラスタリング	k-means	重心の座標（k個ぶん）

3

パラメータは図の中に見えている



- パラメータを変える = 図の中で境界が動く
- 値しだいで正答率が大きく変わる

① ノート

学習とは、パラメータという数字を決めること。どの値が良いかを測る物差しが**損失関数** (loss function)、良い値を探す手段が今日の**最適化**。

4

最適化問題とは

機械学習の中心課題

機械学習では、ほぼすべてのアルゴリズムで**最適化問題**を解く必要があります：

- **ロジスティック回帰**：パラメータを調整して分類精度を最大化
- **ニューラルネットワーク**：重みを調整して予測誤差を最小化
- **サポートベクターマシン**：マージンを最大化する境界を見つける

① 機械学習 = 最適化

「どうパラメータを調整すれば、最も良い結果が得られるか？」を数値的に解く

今回扱う問題

次の関数の最小値を求めます：

$$f(x) = x^2 - 4x + 3$$

数学的には簡単ですが、今回は**数値的に**（機械のやり方で）解きます。

今回の目標

1. **勾配降下法 (gradient descent)** を完全に理解する（手計算 + 実装）
2. 唯一の調整つまみ「**学習率 (learning rate)**」の振る舞いを体感する

5

準備：Pythonのセットアップ

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 # 日本語表示設定（必要に応じて）
6 plt.rcParams['font.size'] = 11
7 plt.rcParams['figure.dpi'] = 100
```

Python

関数の定義

```
1 def f(x):
2     """目的関数  $f(x) = x^2 - 4x + 3$ """
3     return x**2 - 4*x + 3
4
5 def f_prime(x):
6     """1次微分  $f'(x) = 2x - 4$ """
7     return 2*x - 4
```

Python

6

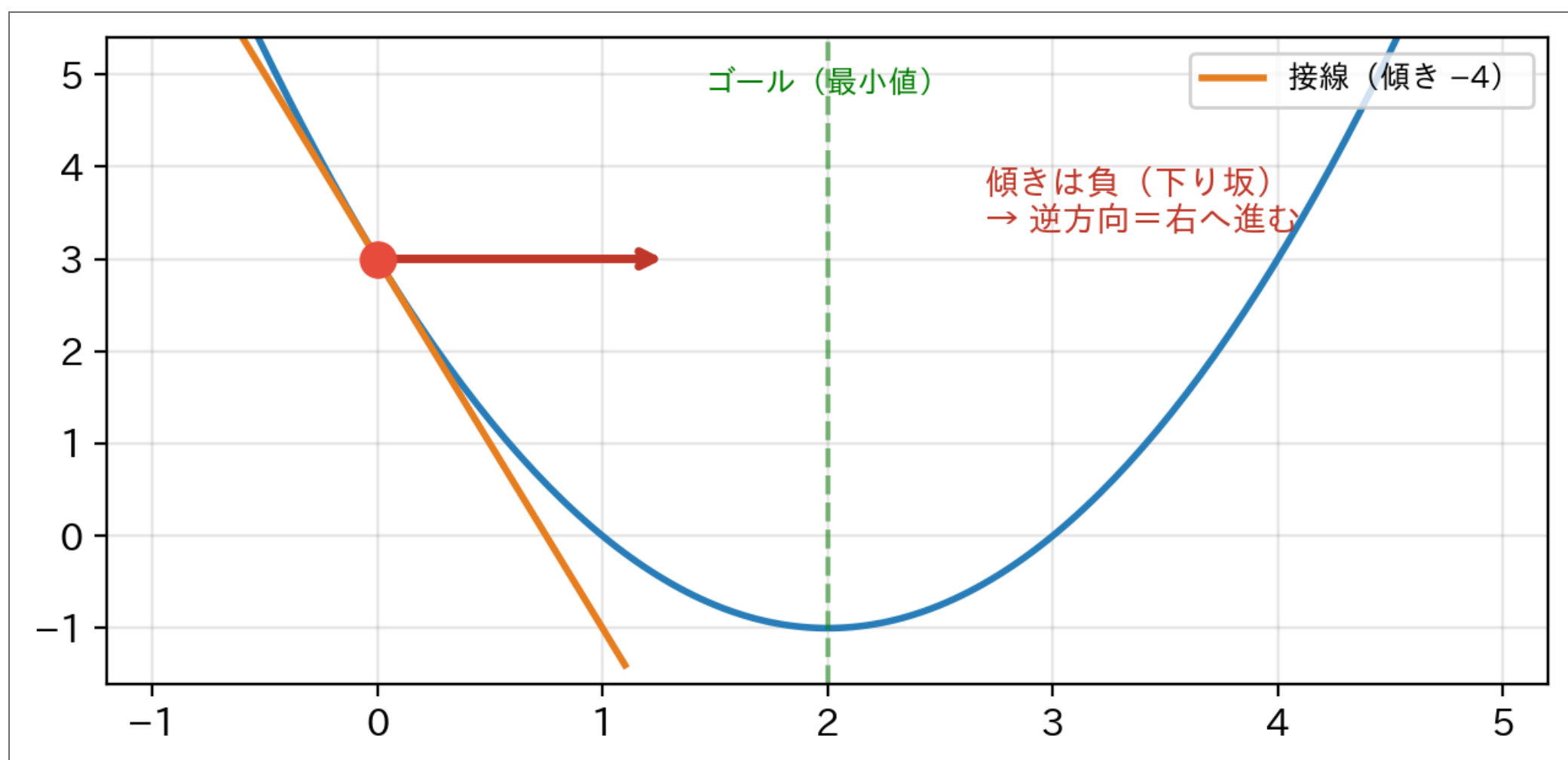
勾配降下法

7

勾配降下法とは

基本的な考え方

1. 現在地点での **傾き（勾配, gradient）** を測る
2. 傾きの **逆方向** に少し進む
3. また傾きを測る
4. 繰り返して最小値を探す



キーワード: 少しずつ、手探りで

アルゴリズム

更新式:

$$x_{\text{new}} = x - \alpha \cdot f'(x)$$

- x : 現在の位置
- $f'(x)$: 勾配（傾き）
- α : **学習率**（どれくらい進むか）
- x_{new} : 次の位置

⚠ 警告

学習率 α は自分で決める必要がある（これが難しい）

勾配降下法：手計算で実行

設定

- 関数: $f(x) = x^2 - 4x + 3$
- 1次微分: $f'(x) = 2x - 4$
- 初期値: $x_0 = 0$
- 学習率: $\alpha = 0.3$

反復1

ステップ1：現在の位置

$$x = 0$$

ステップ2：関数値を計算

$$f(0) = 0^2 - 4 \times 0 + 3 = 3$$

ステップ3：勾配を計算

$$f'(0) = 2 \times 0 - 4 = -4$$

意味: 傾きが -4 （負）なので、右に行くほど値が小さくなる

ステップ4：更新

更新式:

$$x_{\text{new}} = x - \alpha \cdot f'(x)$$

代入:

$$x_{\text{new}} = 0 - 0.3 \times (-4)$$

計算:

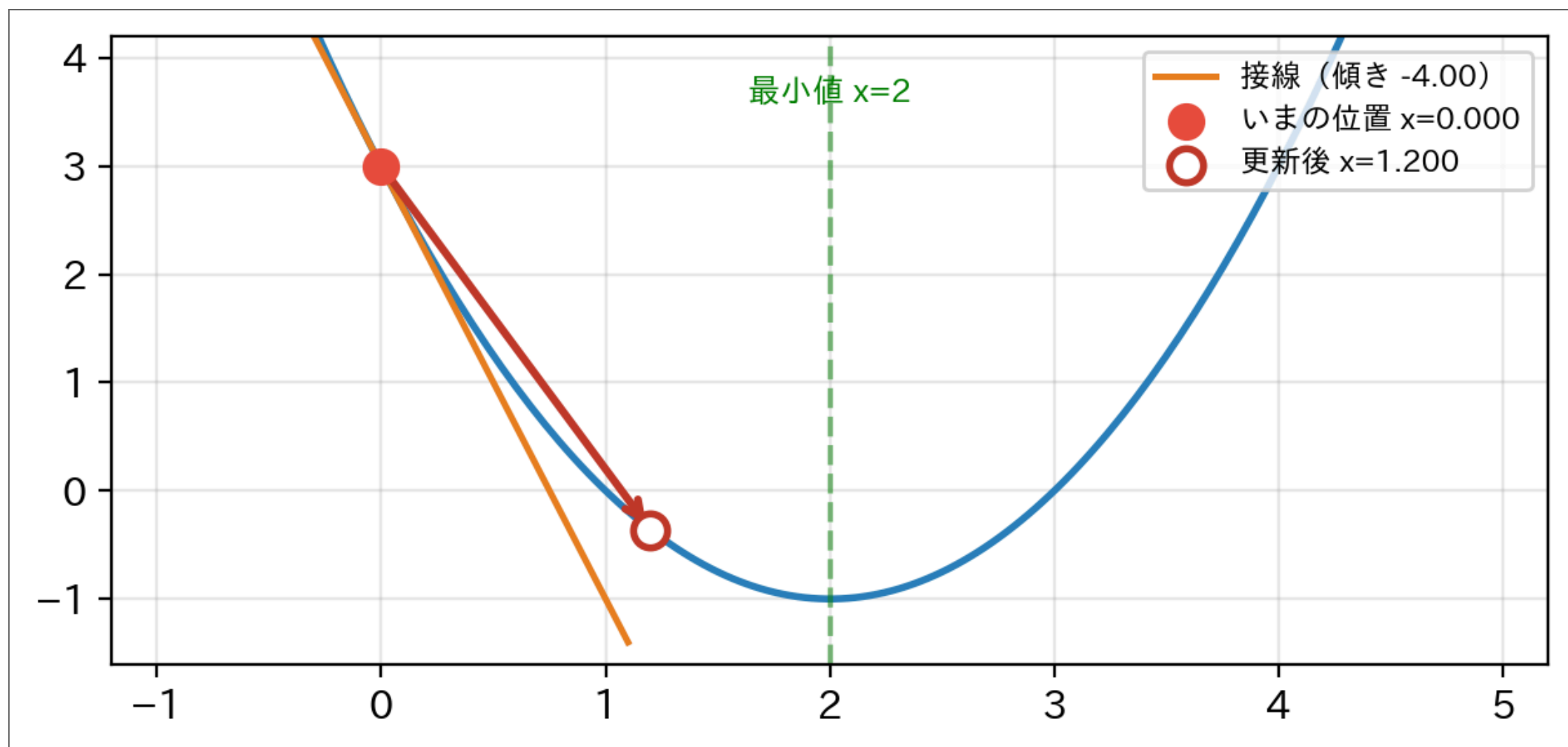
$$= 0 - (-1.2)$$

$$= 0 + 1.2$$

$$= 1.2$$

⚠ 重要

反復1の結果: $x = 0 \rightarrow x = 1.2$



反復2

ステップ1：現在の位置

$$x = 1.2$$

ステップ2：関数値

$$f(1.2) = 1.2^2 - 4 \times 1.2 + 3 = 1.44 - 4.8 + 3 = -0.36$$

ステップ3：勾配

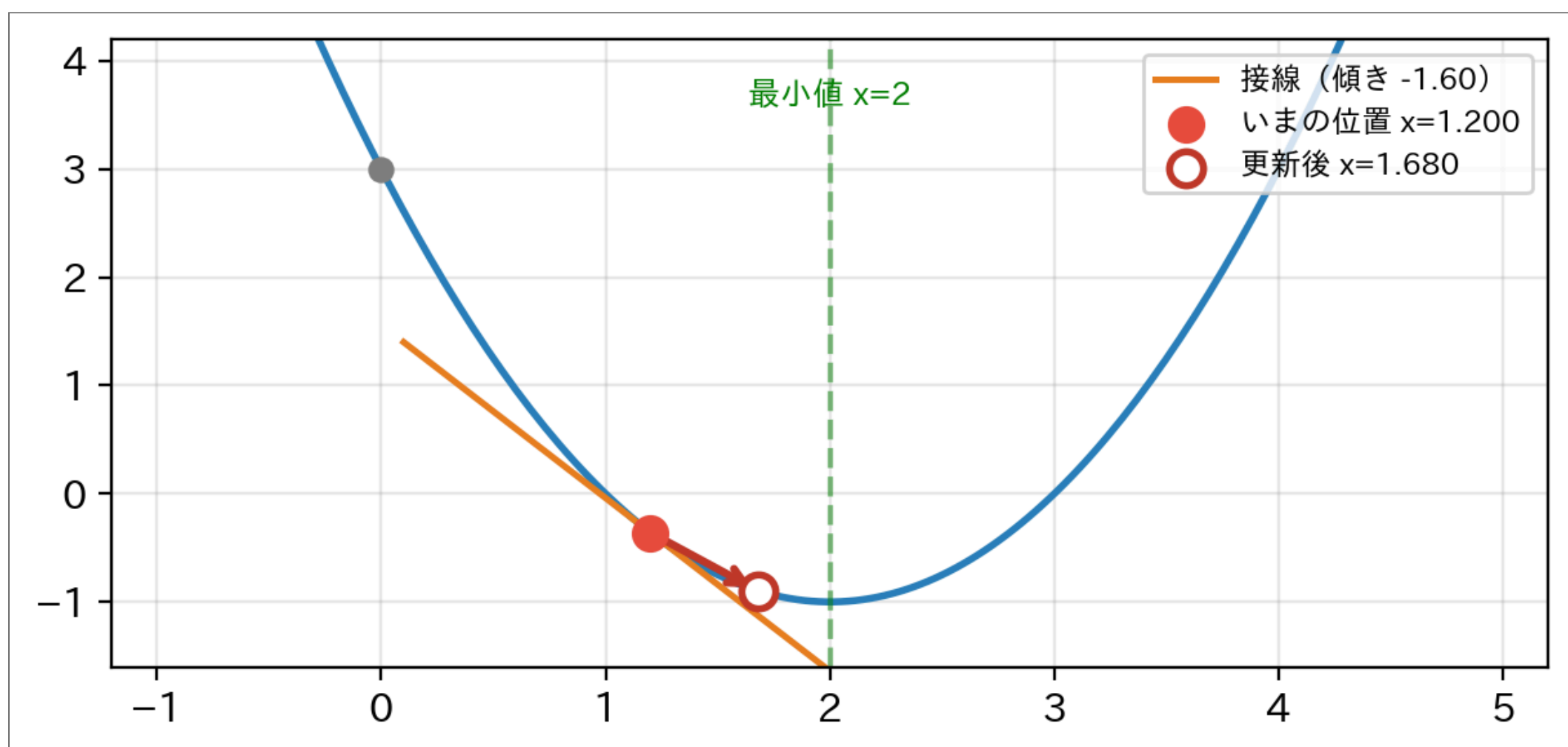
$$f'(1.2) = 2 \times 1.2 - 4 = 2.4 - 4 = -1.6$$

ステップ4：更新

$$x_{\text{new}} = 1.2 - 0.3 \times (-1.6) = 1.2 + 0.48 = 1.68$$

❗ 重要

反復2の結果: $x = 1.2 \rightarrow x = 1.68$



反復3

現在の位置

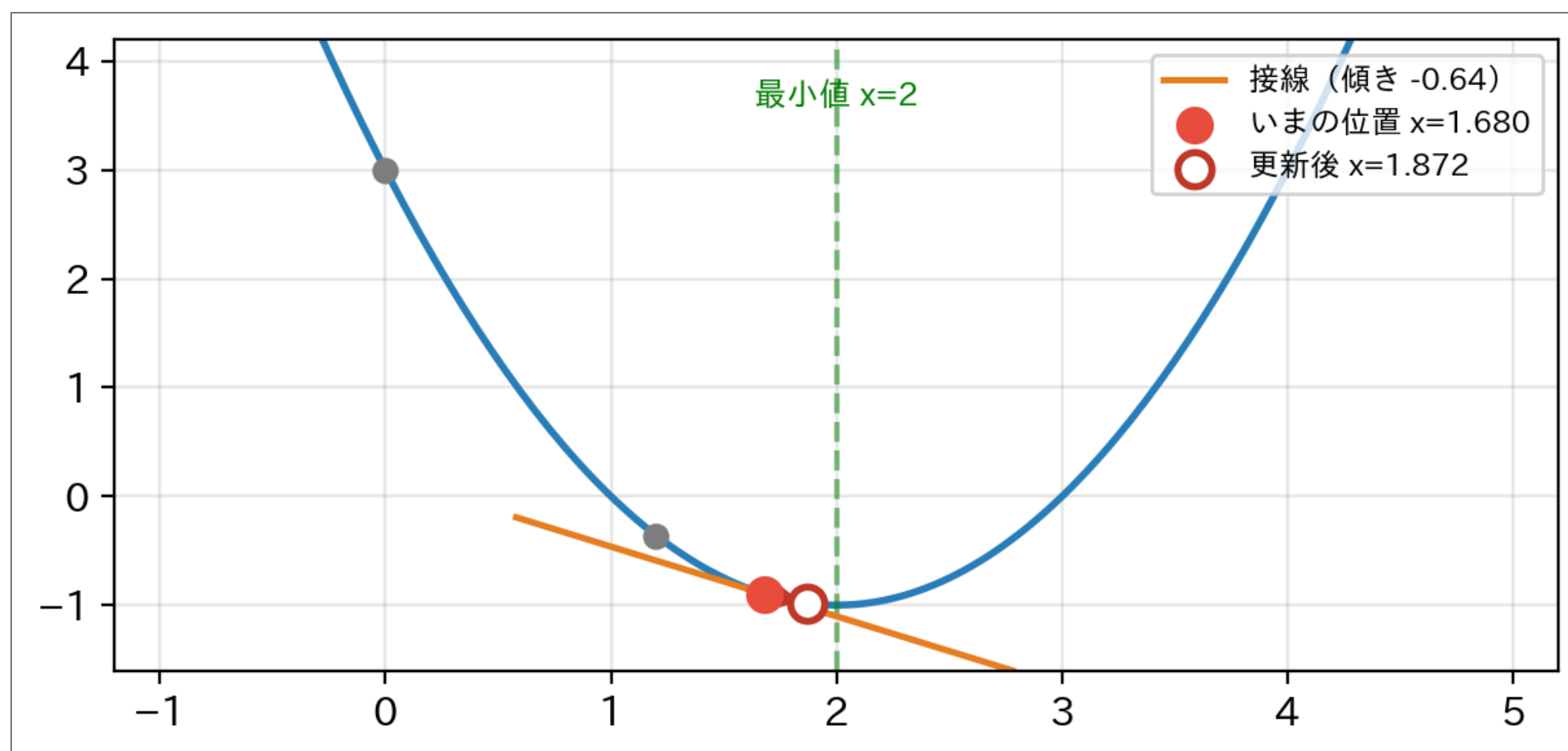
$$x = 1.68$$

勾配

$$f'(1.68) = 2 \times 1.68 - 4 = 3.36 - 4 = -0.64$$

更新

$$x_{\text{new}} = 1.68 - 0.3 \times (-0.64) = 1.68 + 0.192 = 1.872$$



反復4

現在の位置

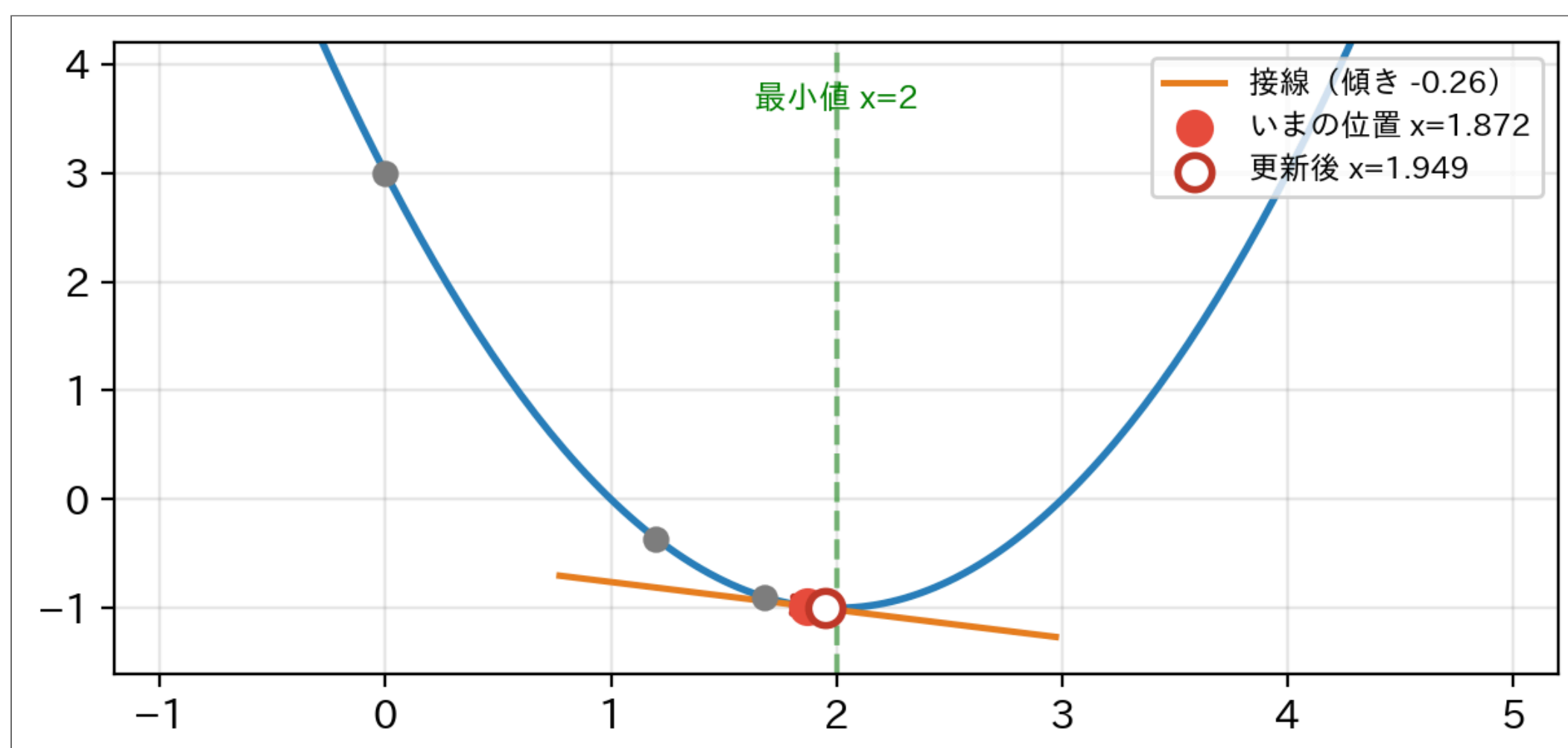
$$x = 1.872$$

勾配

$$f'(1.872) = 2 \times 1.872 - 4 = 3.744 - 4 = -0.256$$

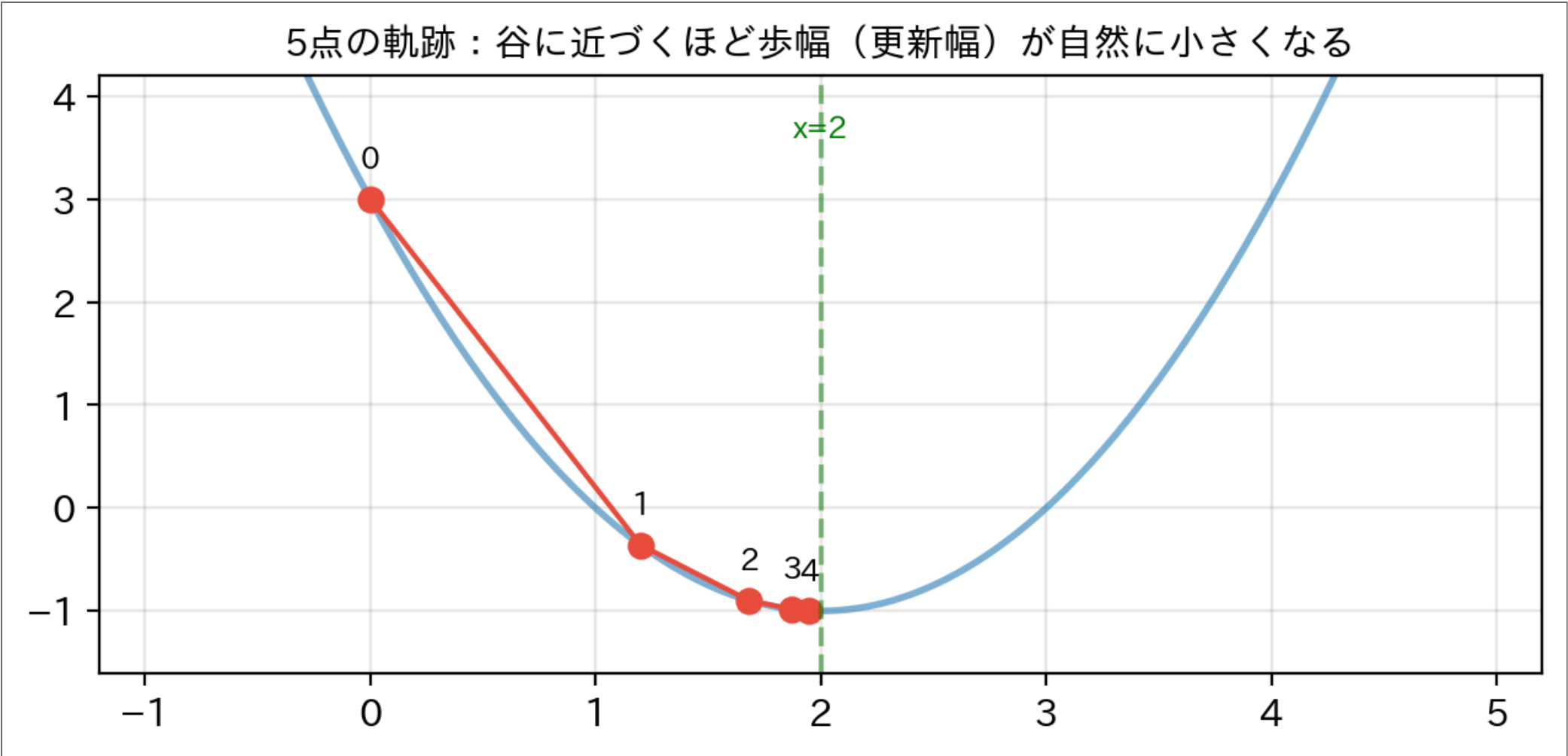
更新

$$x_{\text{new}} = 1.872 - 0.3 \times (-0.256) = 1.872 + 0.0768 = 1.9488$$



進捗まとめ

反復	x	$f(x)$	$f'(x)$	更新幅
0	0.000	3.000	-4.000	-
1	1.200	-0.360	-1.600	+1.200
2	1.680	-0.902	-0.640	+0.480
3	1.872	-0.984	-0.256	+0.192
4	1.949	-0.997	-0.102	+0.077



① ノート

徐々に正解（ $x = 2$ ）に近づいているが、**まだ到達していない**

勾配降下法：Python実装

▶ コードを表示

```
勾配降下法で最小値を求める
=====

--- 反復 1 ---
現在の x = 0.0000
f(x) = 3.0000
f'(x) = -4.0000

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -4.0000 = -1.2000$ 
    x_new = 0.0000 - -1.2000 = 1.2000

--- 反復 2 ---
現在の x = 1.2000
f(x) = -0.3600
f'(x) = -1.6000

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -1.6000 = -0.4800$ 
    x_new = 1.2000 - -0.4800 = 1.6800

--- 反復 3 ---
現在の x = 1.6800
f(x) = -0.8976
f'(x) = -0.6400

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -0.6400 = -0.1920$ 
    x_new = 1.6800 - -0.1920 = 1.8720

--- 反復 4 ---
現在の x = 1.8720
f(x) = -0.9836
f'(x) = -0.2560

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -0.2560 = -0.0768$ 
    x_new = 1.8720 - -0.0768 = 1.9488

--- 反復 5 ---
現在の x = 1.9488
f(x) = -0.9974
f'(x) = -0.1024

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -0.1024 = -0.0307$ 
    x_new = 1.9488 - -0.0307 = 1.9795

--- 反復 6 ---
現在の x = 1.9795
f(x) = -0.9996
f'(x) = -0.0410

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -0.0410 = -0.0123$ 
    x_new = 1.9795 - -0.0123 = 1.9918

--- 反復 7 ---
現在の x = 1.9918
f(x) = -0.9999
f'(x) = -0.0164

更新：
    ステップ幅 =  $\alpha \times f'(x) = 0.3 \times -0.0164 = -0.0049$ 
    x_new = 1.9918 - -0.0049 = 1.9967

--- 反復 8 ---
現在の x = 1.9967
```

$f(x) = -1.0000$
 $f'(x) = -0.0066$

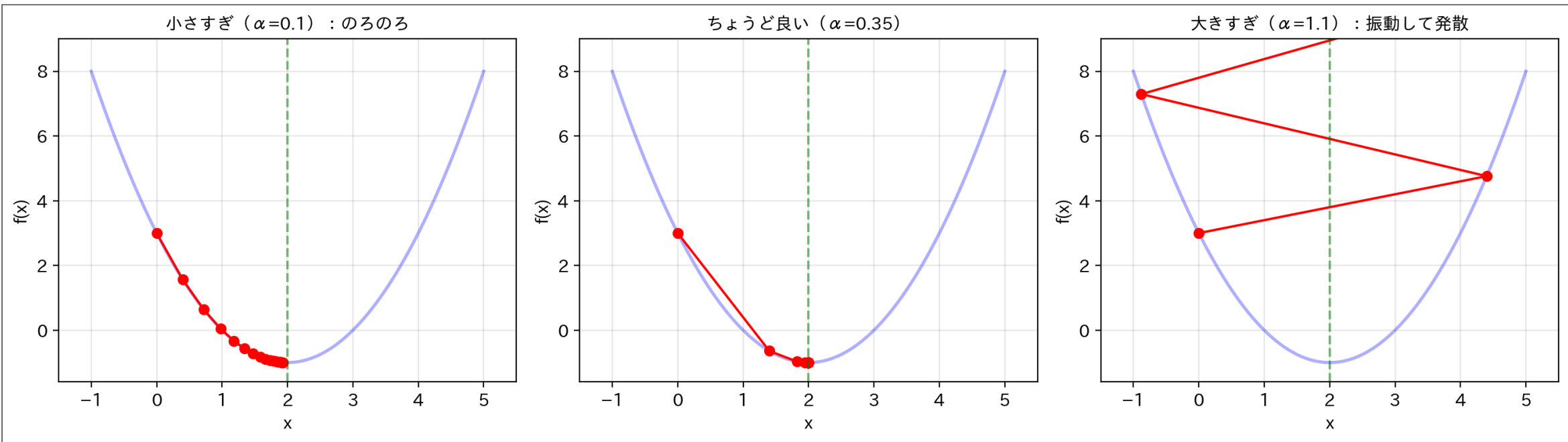
更新:
ステップ幅 = $\alpha \times f'(x) = 0.3 \times -0.0066 = -0.0020$
 $x_{\text{new}} = 1.9967 - -0.0020 = 1.9987$

★ ほぼ収束 ($f'(x) < 0.01$)

=====
最終結果: $x = 1.9987, f(x) = -1.0000$
理論値との誤差: 0.001311

勾配降下法の問題点

学習率の選択が難しい



⚠ 警告

- α が小さすぎる: 収束が遅い
- α が大きすぎる: 振動して収束しない
- 適切な α を探すのが難しい

収束が遅い

10回反復してもまだ誤差がある。複雑な問題では数百～数千回必要になることも。

それでも主役は勾配降下法

- 弱点（学習率の調整・遅さ）があるのに、**ディープラーニングのエンジンは勾配降下法**
- 理由：パラメータが**数百万個**あっても、1次微分さえ計算できれば動く（軽い）
- 学習率は、自動で加減する改良版（Adam など）が実用では使われている

i 参考資料

- もっと賢く進む手法もある → **Newton法** [↗](#)（2次微分で一気にジャンプ）
- 谷が複数あるとき、どの谷に落ちるかは初期値しだい → **ローカルミニマムの罠** [↗](#)

まとめ

- **最適化** = パラメータを動かして、損失が最小の場所を探すこと (**fit()** の正体)
- **勾配降下法** : 傾きを測り、逆方向に少し進む、を繰り返す

$$x_{\text{new}} = x - \alpha \cdot f'(x)$$

- **学習率** α が唯一の調整つまみ : 小さすぎると遅い、大きすぎると発散
- シンプルで軽いからこそ、**大規模問題 (ディープラーニング) の標準エンジン**

13

次回予告

- 今日手に入れた勾配降下法が、**fit()** の中身を開ける鍵になる
- 次回、**本物のモデル (線形回帰)** を、この道具で自分の手で **fit** する
- その先に分類モデル、そしてニューラルネットワーク — 勾配降下法は**ディープラーニングを動かしている本物のエンジン**

14

練習問題

以下の問題を **手計算** と **Pythonコード** の両方で解け。

問題1 : 勾配降下法

関数 $f(x) = x^2 + 6x + 5$ について :

1. 1次微分を求めよ
2. 初期値 $x_0 = 0$ 、学習率 $\alpha = 0.2$ で勾配降下法を実行せよ
3. 反復1、反復2、反復3を手計算で示せ

発展課題 : **Newton法 (参考資料)** の練習問題で、同じ関数を別の手法で解いて速さを比較する。

15

問題1の解答例

準備

$$f(x) = x^2 + 6x + 5$$

1次微分:

$$f'(x) = 2x + 6$$

反復1

現在: $x = 0$

$$f'(0) = 2(0) + 6 = 6$$

更新:

$$x_{\text{new}} = 0 - 0.2 \times 6 = 0 - 1.2 = -1.2$$

反復2

現在: $x = -1.2$

$$f'(-1.2) = 2(-1.2) + 6 = -2.4 + 6 = 3.6$$

更新:

$$x_{\text{new}} = -1.2 - 0.2 \times 3.6 = -1.2 - 0.72 = -1.92$$

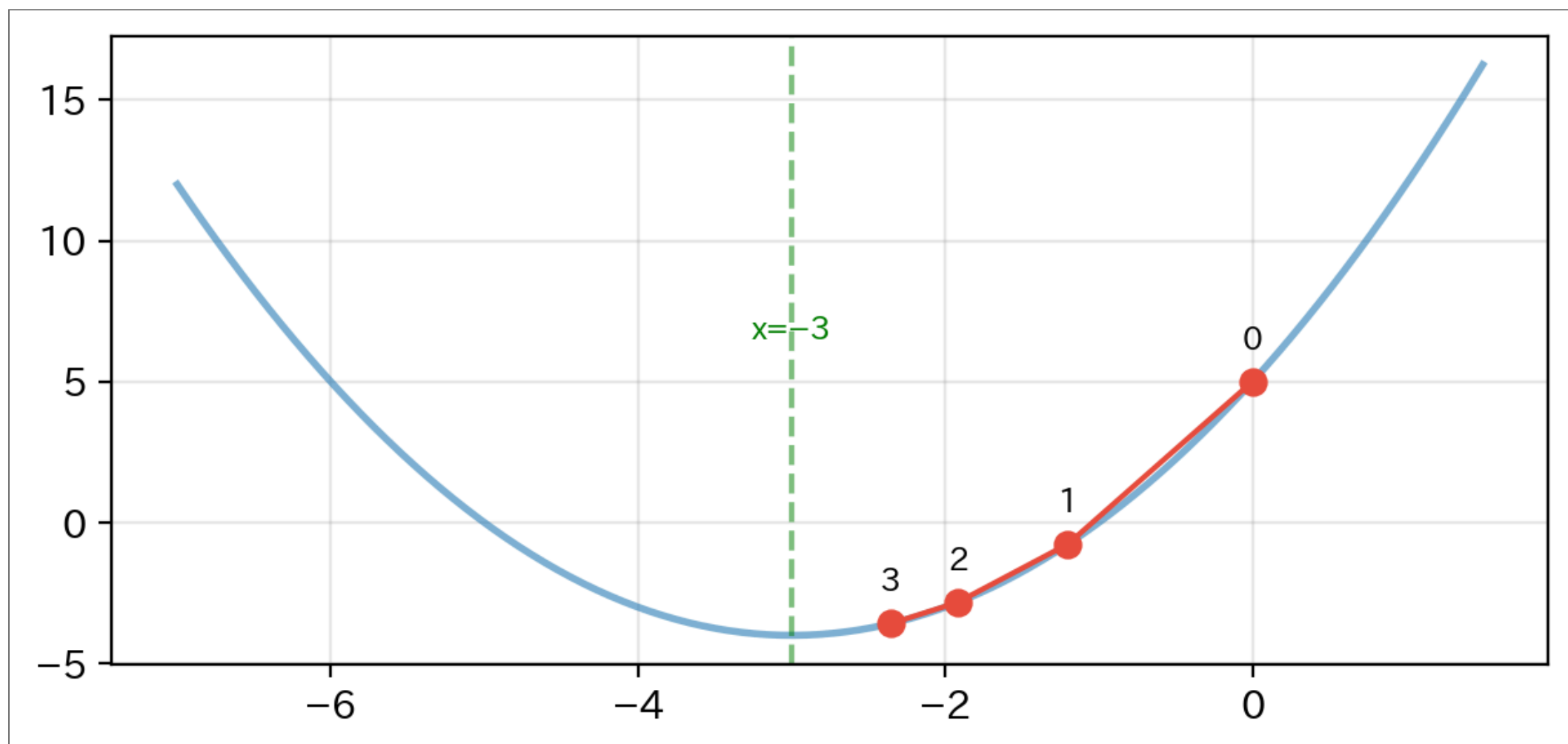
反復3

現在: $x = -1.92$

$$f'(-1.92) = 2(-1.92) + 6 = -3.84 + 6 = 2.16$$

更新:

$$x_{\text{new}} = -1.92 - 0.2 \times 2.16 = -1.92 - 0.432 = -2.352$$



まだ収束していない (理論値は $x = -3$)

教科書

- Stephen Boyd & Lieven Vandenberghe, "Convex Optimization" (2004)
- Jorge Nocedal & Stephen Wright, "Numerical Optimization" (2006)

日本語資料

- 杉山将「イラストで学ぶ機械学習」(2013)
- 金谷健一「これなら分かる最適化数学」(2005)

オンライン資料

- [scipy.optimize documentation](#) 
- [scikit-learn: Logistic Regression](#) 